

Ok, por favor realiza el diseño de la Arquitectura Técnica Real (CTO)

Perfecto. Voy a proponerte una **Arquitectura Técnica Real**, pensada no como prototipo escolar, sino como un **producto SaaS serio**, alineado con lo que ya vienes construyendo en **AppWDFCR**.

1. Objetivo técnico de la arquitectura

La plataforma debe resolver tres flujos principales: **ingesta de CFDI desde correo, extracción estructurada de datos y analítica de negocio**. Como base técnica, tiene mucho sentido usar **Gmail API con OAuth 2.0** para acceso autorizado al correo, un backend API en **FastAPI**, un sistema de tareas asíncronas con **Celery**, y almacenamiento separado entre **base de datos transaccional y object storage** para PDFs/XML/ZIPs. Google documenta el acceso por Gmail API y sus scopes; FastAPI documenta tareas posteriores a la respuesta y despliegue; Celery documenta colas de tareas distribuidas; y Supabase documenta Storage y Postgres administrado como bloques de plataforma. ([Google for Developers](#))

2. Decisión de arquitectura: monolito modular primero, no microservicios desde el día 1

Mi recomendación real es arrancar con un **monolito modular** y no con microservicios. Es decir: un solo backend principal, pero dividido internamente por dominios bien definidos. Para tu caso, eso reduce complejidad operativa, acelera desarrollo y sigue siendo escalable si desde el inicio separas: API, workers, base de datos, cache/cola y almacenamiento de archivos. FastAPI encaja bien en este enfoque porque permite crecer desde una API compacta hacia despliegues más robustos sin obligarte desde el principio a fragmentar todo el sistema. ([FastAPI](#))

Dominios internos del backend

1. **Auth & Tenant**
2. **Email Ingestion**
3. **CFDI Parser**
4. **Extraction Config Engine**
5. **Batch Processing**
6. **Analytics Engine**
7. **Reporting & Export**
8. **Audit & Observability**

3. Arquitectura lógica de alto nivel

La arquitectura quedaría así:

Frontend React / Next.js

→ **API Gateway / Backend FastAPI**

→ **Servicios internos del monolito**

→ **Celery Workers** para procesos pesados

→ **Redis** como broker/cache

→ **PostgreSQL** como base principal

→ **Object Storage** para archivos PDF/XML/ZIP/Excel

→ **Motor analítico** para KPIs y agregados

→ **Dashboards y exportación**

Esto sigue un patrón muy sano: la API atiende rápido, y todo lo pesado se manda a cola. FastAPI documenta tareas posteriores a la respuesta; Celery documenta el modelo de colas y workers dedicados; y su documentación también cubre uso con Redis como broker. ([FastAPI](#))

4. Componentes técnicos concretos

4.1 Frontend

Stack recomendado

- **Next.js + React**
- **TypeScript**
- **Tailwind CSS**
- **TanStack Table**
- **Recharts o Apache ECharts**
- **React Query / TanStack Query**
- **Zustand** para estado liviano

Responsabilidad del frontend

El frontend no debe procesar archivos pesados ni lógica fiscal. Su responsabilidad debe ser:

- autenticación y sesión
- carga visual de facturas muestra
- selección de campos
- visualización de jobs
- dashboards y filtros
- descarga de reportes

Pantallas clave

- Login / selección de cuenta Google
- Dashboard principal
- Bandeja de sincronización de correos
- Constructor visual de JSON de extracción
- Procesamiento masivo ZIP/carpeta
- Dashboard analítico
- Configuración por empresa

4.2 Backend principal

Stack recomendado

- **FastAPI**
- **Pydantic**
- **SQLAlchemy 2.x**
- **Alembic**
- **Uvicorn/Gunicorn** en producción

Responsabilidades

- exponer API REST
- validar sesiones y permisos
- registrar jobs
- orquestar workflows
- servir metadatos de archivos
- generar reportes descargables
- exponer datasets y KPIs

Organización interna

app/
api/
core/
auth/
tenants/
gmail/
cfdi/
extraction/

batch/
analytics/
reports/
audit/
db/
schemas/
workers/

4.3 Procesamiento asíncrono

Aquí está una de las decisiones más importantes.

Recomendación

Usa **Celery + Redis** desde la primera versión seria.

Por qué

Porque tu sistema tiene tareas naturalmente asíncronas:

- leer miles de correos
- descargar adjuntos
- desempaquetar ZIPs
- parsear cientos o miles de XML/PDF
- recalcular indicadores
- generar Excel/PDF

FastAPI tiene Background Tasks, pero están pensadas para trabajos simples posteriores a una respuesta. Para procesamiento pesado, Celery encaja mejor porque su propósito es precisamente distribuir trabajo entre workers mediante cola y broker; además documenta compatibilidad con Redis. ([FastAPI](#))

Colas sugeridas

- email_sync
- attachment_download
- cfdi_parse
- zip_batch
- analytics_refresh
- report_export

Workers separados

- Worker de ingesta
- Worker de parsing
- Worker analítico
- Worker de exportación

Eso te permite escalar solo lo que duele.

4.4 Integración con Gmail / Google Workspace

Integración recomendada

Usa **Gmail API**, no scraping de interfaz ni IMAP como mecanismo principal.

Google describe Gmail API como la opción adecuada para acceso autorizado a buzones Gmail en aplicaciones web; además documenta scopes específicos para lectura y acceso a adjuntos. ([Google for Developers](#))

Flujo

1. Usuario inicia sesión con Google OAuth
2. Se guarda token cifrado
3. Usuario define filtros:

- remitente
 - rango de fechas
 - palabras clave
 - con/sin adjuntos
4. Se lanza job de sincronización
 5. Se listan mensajes
 6. Se descargan adjuntos PDF/XML
 7. Se vinculan en una entidad documental

Scopes mínimos

Para solo lectura, la arquitectura debe buscar el menor privilegio posible. Google documenta que distintos métodos de Gmail requieren scopes como gmail.readonly o gmail.modify, dependiendo de la operación exacta. ([Google for Developers](#))

Regla arquitectónica importante

Nunca mezclar lógica de Gmail con lógica contable.

El módulo Gmail solo trae mensajes, adjuntos y metadatos.

El dominio CFDI decide si eso es una factura válida o no.

4.5 Base de datos

Recomendación real

Usa **PostgreSQL** como base principal.

Aunque originalmente habías considerado MySQL, aquí PostgreSQL te da mejor flexibilidad para:

- JSON/JSONB
- vistas materializadas
- consultas analíticas más cómodas
- modelado más limpio para configuraciones y metadatos

Si ya llevas avance fuerte en MySQL, puedes iniciar ahí. Pero si estás a tiempo, para este producto yo movería la arquitectura a PostgreSQL.

Qué guardar en DB

No guardes los PDF/XML completos dentro de la base.

Guarda:

- usuarios
- empresas/tenants
- cuentas conectadas
- mensajes sincronizados
- adjuntos y rutas
- CFDIs normalizados
- conceptos/partidas
- configuración JSON
- jobs
- métricas y snapshots analíticos
- auditoría

Esquema conceptual mínimo

users

tenants

user_tenant_roles

email_accounts

email_messages

email_attachments

cfdi_documents
cfdi_parties
cfdi_items
extraction_templates
processing_jobs
analytics_snapshots
export_files
audit_logs

4.6 Almacenamiento de archivos

Recomendación

Usa **object storage**, no disco local del servidor.

Puedes usar:

- **Supabase Storage**
- S3 compatible
- MinIO en local/dev

Supabase documenta Storage como almacenamiento escalable para archivos y además documenta que cada proyecto usa una base Postgres y políticas de acceso integradas. ([Supabase](#))

Estructura de buckets

- raw-email-attachments
- normalized-cfdi
- zip-uploads
- generated-reports
- temp-processing

Convención de rutas

tenant/{tenant_id}/year/{yyyy}/month/{mm}/source/{provider}/file.ext

Regla importante

Guardar dos versiones:

- **raw**: archivo original
- **processed**: archivo ya validado/normalizado

Eso te da trazabilidad.

4.7 Motor CFDI y parser documental

Este es el corazón del producto.

Submódulos internos

- xml_parser
- pdf_parser
- cfdi_validator
- field_mapper
- template_engine
- normalizer

Estrategia técnica correcta

1. **El XML es la fuente de verdad estructurada**
2. **El PDF es complemento visual / respaldo**
3. Si existe conflicto, gana el XML salvo reglas explícitas de negocio

Pipeline de parsing

1. Detectar tipo de archivo
2. Si es ZIP, expandir

3. Vincular PDF ↔ XML por UUID/nombre/fecha/importe
4. Parsear XML a estructura estándar
5. Extraer texto/tablas del PDF cuando aplique
6. Comparar campos detectados
7. Generar vista de selección de campos
8. Aplicar plantilla JSON
9. Normalizar dataset de salida

Normalización clave

Tu sistema no debe depender de “cómo viene escrita” la factura.

Debe producir un modelo canónico:

- emisor
- receptor
- fecha
- uuid
- moneda
- subtotal
- impuestos
- total
- conceptos
- cantidades
- precio_unitario
- sku/clave
- descripción
- costo
- margen estimado

4.8 Motor de plantillas JSON

Este módulo debe volverse una capacidad estratégica del producto.

Función

Permitir que el usuario “enseñe” al sistema qué campos quiere extraer.

Diseño recomendado

Guardar plantillas versionadas.

Ejemplo conceptual:

```
{
  "template_version": 3,
  "name": "ventas_producto_cliente",
  "source_type": "cfdi",
  "fields": [
    {"key": "fecha", "source": "xml", "path": "Comprobante.Fecha"},
    {"key": "cliente", "source": "xml", "path": "Receptor.Nombre"},
    {"key": "producto", "source": "xml", "path": "Conceptos[.Descripcion"],
    {"key": "cantidad", "source": "xml", "path": "Conceptos[.Cantidad"],
    {"key": "precio", "source": "xml", "path": "Conceptos[.ValorUnitario"],
    {"key": "total", "source": "xml", "path": "Comprobante.Total"}
  ]
}
```

Regla técnica importante

No guardes solo una lista de nombres; guarda:

- nombre de campo
- fuente
- path
- tipo de dato
- formato
- transformaciones
- si es obligatorio
- si acepta nulos

Eso te permitirá evolucionar luego a un verdadero **ETL configurable**.

4.9 Motor analítico

El tercer módulo merece diseño propio.

Enfoque recomendado

Separar:

- **modelo transaccional**
- **modelo analítico**

No hagas todos los KPIs directamente sobre tablas crudas si el volumen crece.

Diseño sugerido

Crear tablas agregadas o snapshots:

- ventas por mes
- compras por mes
- utilidad por producto
- utilidad por cliente
- top productos
- top clientes
- series temporales

Dos estrategias posibles

Opción A: Analítica dentro de PostgreSQL

Buena para MVP y etapa inicial.

- vistas
- materialized views
- tablas agregadas refrescadas por jobs

Opción B: Data warehouse ligero posterior

Cuando crezcas:

- ClickHouse
- BigQuery
- DuckDB para pipelines offline
- dbt más adelante

Mi recomendación real

Empieza con **PostgreSQL + tablas agregadas + refresh asíncrono**.

No metas warehouse todavía.

4.10 Exportación y reporting

Salidas del sistema

- Excel
- CSV
- PDF de reportes
- dashboards web

Diseño correcto

La exportación también debe ir a cola.

Generar reportes grandes sin bloquear la API.

Flujo

1. Usuario pide reporte
2. Se crea processing_job
3. Worker genera archivo
4. Se guarda en bucket
5. Se notifica al frontend
6. Usuario descarga

5. Arquitectura de datos

5.1 Modelo por capas

Capa 1: Raw

Archivos sin tocar:

- correos
- XML
- PDF
- ZIP

Capa 2: Parsed

Datos ya extraídos:

- CFDI header
- conceptos
- emisor/receptor
- impuestos

Capa 3: Canonical

Modelo estándar del negocio:

- cliente
- proveedor
- producto
- compra
- venta
- margen
- periodo

Capa 4: Analytics

KPIs y agregados:

- ventas mensuales
- ranking productos
- ranking clientes
- utilidad

Este patrón te ahorra mucho dolor futuro.

6. Seguridad real

6.1 Principios

- mínimo privilegio
- cifrado de tokens
- separación por tenant
- auditoría completa

- URLs firmadas para descarga
- caducidad de sesiones

Google documenta que los scopes deben declararse según el nivel real de acceso requerido, y algunos son sensibles; por eso conviene pedir el menor alcance necesario para lectura. ([Google for Developers](#))

6.2 Qué proteger especialmente

- refresh tokens de Google
- correos sincronizados
- RFCs y datos fiscales
- reportes exportados
- plantillas JSON

6.3 Medidas concretas

- cifrado de tokens en reposo
- secretos fuera del código
- RLS o equivalente por tenant
- logs de acceso
- expiración de links de descarga
- hash y trazabilidad de archivos

7. Multiempresa / multitenancy

Como esto apunta a producto SaaS, debes diseñar multitenancy desde hoy.

Recomendación

Single database, shared schema, tenant_id en todas las entidades de negocio, con aislamiento fuerte a nivel aplicación y, si usas Postgres/Supabase, políticas de acceso por fila donde corresponda. Supabase documenta integración entre base Postgres, APIs y políticas de acceso para datos/archivos. ([Supabase](#))

No recomiendo al inicio

- una base por cliente
- un deployment por cliente

Eso complica demasiado operación y costos.

8. Observabilidad y operación

Logs

Debes registrar:

- login y refresh de tokens
- sync jobs
- archivos descargados
- errores de parsing
- conflictos PDF/XML
- exportaciones
- accesos a reportes

Métricas

- correos procesados por minuto
- adjuntos descargados
- tasa de error de parsing
- tiempo por job
- tiempo de generación de Excel
- uso por tenant

- productos/consultas más utilizadas

Trazabilidad

Cada job debe tener:

- id
- tenant
- usuario que lo disparó
- input
- estado
- timestamps
- errores
- output artifacts

9. Despliegue real

Entornos

- local
- staging
- production

Producción mínima recomendable

- Frontend en Vercel o similar
- Backend FastAPI en contenedor
- Worker Celery en contenedor separado
- Redis administrado
- PostgreSQL administrado
- Storage administrado
- monitoreo básico

FastAPI documenta distintas estrategias de despliegue y resalta temas como HTTPS, reinicios, replicación y memoria al decidir la configuración productiva. ([FastAPI](#))

Topología mínima

- frontend-web
- api-server
- worker-ingestion
- worker-processing
- worker-analytics
- redis
- postgres
- storage

10. Escalabilidad por etapas

Etapas 1 – MVP serio

- 1 API
- 1 worker general
- Postgres
- Redis
- Storage
- dashboards básicos

Etapas 2 – Producto vendible

- workers separados por tipo
- tablas agregadas

- notificaciones de jobs
- control de cuotas por tenant
- auditoría formal

Etapas 3 – SaaS maduro

- autoscaling de workers
- colas por prioridad
- warehouse analítico
- forecasting
- recomendación de compras/ventas
- alertas inteligentes

11. Arquitectura final recomendada para tu caso

Si yo fuera tu CTO, te recomendaría esta versión 1 sería:

Frontend

- Next.js
- TypeScript
- Tailwind
- React Query

Backend

- FastAPI
- SQLAlchemy
- Alembic
- Pydantic

Async

- Celery
- Redis

Data

- PostgreSQL

Files

- Supabase Storage o S3 compatible

Integraciones

- Gmail API + OAuth 2.0

Analytics

- PostgreSQL agregados + snapshots

Deploy

- contenedores separados para API y workers

Estrategia de arquitectura

- monolito modular
- multitenant
- event-driven en procesos pesados
- object storage para documentos
- modelo canónico CFDI
- analítica desacoplada del parsing

12. Mi conclusión técnica honesta

Para tu proyecto, **la arquitectura correcta no es “hacer una app que lea correos”**.

La arquitectura correcta es construir una plataforma con cuatro capacidades centrales:

1. **ingesta confiable**
2. **normalización fiscal/documental**
3. **pipeline masivo de datos**
4. **capa analítica para decisión de negocio**

Eso convierte tu idea en algo mucho más fuerte: una especie de **sistema híbrido entre extractor CFDI, ETL financiero y BI para PyMEs**.

El siguiente paso natural es bajar esto a un nivel todavía más ejecutable:

diagrama de componentes + modelo de datos + roadmap técnico por sprints.